

# **ESI2026 - Car Pooling System**

Renan Barbosa de Moraes, Tomáš Magula,  
Paulius Sabanskis, Simon Schwitz

April 19, 2026

# 1 Realistic system

The proposed system is a comprehensive carpooling system designed to connect people traveling in the same direction. It facilitates journey sharing by allowing drivers to publish upcoming trips from point A to point B, while enabling passengers to seamlessly search for, reserve, and pay for seats on those trips. The system is used by two primary user types: drivers and passengers. Drivers register, create profiles, undergo identity and vehicle verification, publish their routes, communicate with passengers, and verify payments after a successful drop-off. Passengers register, create profiles, search for available rides to their destination, reserve seats, pay for the service, and leave feedback for their drivers.

The platform delivers several core functionalities essential for a complete ride-sharing experience. Ride management allows drivers to create rides by specifying a vehicle, start and end locations, a start time, and a price, while allowing passengers to make reservations. The search and discovery functionality provides an engine that finds the best possible rides based on passenger destination, driver ratings, past rides, and distance, using geocoding capabilities. Trust and verification functionalities validate user IDs, driver's licenses, and vehicle registration data to ensure platform safety. Secure payment processing handles financial transactions for completed rides. Finally, a reputation system records reviews and star ratings from both drivers and passengers to build community trust.

A monolithic architecture wouldn't make sense for this platform because the components have distinct scaling, security, and integration requirements. For example, the geolocation service needs to handle heavy traffic from continuous location autocomplete queries. By isolating it, we can scale our map-related compute resources without impacting the core ride-booking process. Security is another major factor. The authentication service needs strict isolation so that no other part of the system ever stores sensitive data, such as a user's password. Similarly, keeping payment processing and document validation separate allows us to securely outsource these tasks to third-party providers such as Stripe, PayPal, or external identity verification services. This guarantees that no other internal services can access users' restricted financial or regulatory documents. Also, decoupling the system means that if a secondary feature, such as the review service, goes down, it won't affect day-to-day operations. Passengers can still seamlessly book, take, and pay for their rides even if they can't leave a star rating at that exact moment.

## 2 Architecture

The Car Pooling System's logical architecture relies on eight distinct microservices. The core experience is driven by the Ride Booking and Search Engine services, which allow users to create, discover, and book trips. Transactions and user feedback are managed seamlessly by the dedicated Payment and Review services. Meanwhile, the Profile, Authentication, Validation, and Geolocation services work securely behind the scenes to manage accounts, verify legal documents, control access, and calculate route distances.

### 2.1 Ride Booking Service

**Responsible:** Paulius

The responsibility of the Ride Booking Service is to create and book rides. This service allows drivers to create rides from point A to point B and enables passengers to book. It strictly handles data related to rides and bookings. It operates as a standalone service because its sole goal is to manage rides and associated bookings without needing to concern itself with how search or payment functionality is implemented. To function securely, it must validate the user's identity with the Authentication service. It depends on the Payment service to manage financial transactions for rides, and on the Profile Service to determine which vehicles are available for a driver to select when creating a ride. Additionally, it relies on the Geolocation Service to geocode the start and end positions of the rides.

### 2.2 Ride Discovery Service

**Responsible:** Tomáš

The Ride Search Engine is tasked with finding the best possible ride options for a passenger from all available choices. This service does not store its own primary data; instead, it aggregates and filters data retrieved from other services. It operates independently to ensure that ride search operations do not negatively affect existing bookings. To operate effectively, it depends on the Ride Booking service to find available rides. It relies on the Profile and

Review services because search results are influenced by past rides, driver ratings, and user feedback. Also, it must validate the user's identity with the Authentication service.

## 2.3 Payment Service

**Responsible:** Tomáš

The Payment Service is responsible for handling the entire payment process after a ride is complete. It exclusively manages payment transaction data. This service is isolated into its own bounded context because payment processing can be safely outsourced to external providers, such as Stripe or PayPal.

## 2.4 Review Service

**Responsible:** Paulius

The Review Service is responsible for recording reviews for rides taken. It manages data consisting of user reviews, including star ratings and written comments. This service is separated from the core system because reviews are a secondary functionality and not strictly necessary for day-to-day operations. It depends on the Ride Booking service to verify and allow passengers to leave their reviews, and on the Authentication service to validate users' identity.

## 2.5 Profile Service

**Responsible:** Simon

The Profile Service manages all personal information for system users. For drivers, this data specifically includes their driver's license state and details regarding all their vehicles. The service manages the user profile data, including driver statuses and vehicle information. It is structured as a standalone service because user profile data is heavily used across multiple services, and centralizing it effectively eliminates the need for each service to duplicate it. It depends on the Authentication service to validate users' identity. To maintain verified records, it actively communicates with the driver and vehicle Validation service.

## 2.6 Authentication Service

**Responsible:** Simon

The Authentication Service is responsible for user authentication, verifying that users are who they claim to be, and managing user authorization roles, such as distinguishing between drivers and passengers. The data it handles includes user sessions and role permissions. It operates independently because user authorization is used by multiple services, and isolating it prevents data duplication across the system. Furthermore, it is separated for security reasons, ensuring that no other service stores sensitive information, such as a user's password. It is a dependency for all other user-facing microservices that handle user input, so they can verify that the input is legitimate and originates from the correct user.

## 2.7 Validation Service

**Responsible:** Renan

The Validation Service is responsible for validating a user's ID, driver's license, and vehicle registration data. The data it manages includes regulatory mandatory documents, such as driver's licenses and vehicle documents. It remains an independent service because document verification could easily be outsourced to different providers, and nobody else should have access to these highly sensitive documents. If the validation is successful, the verified information will be stored in the user's profile.

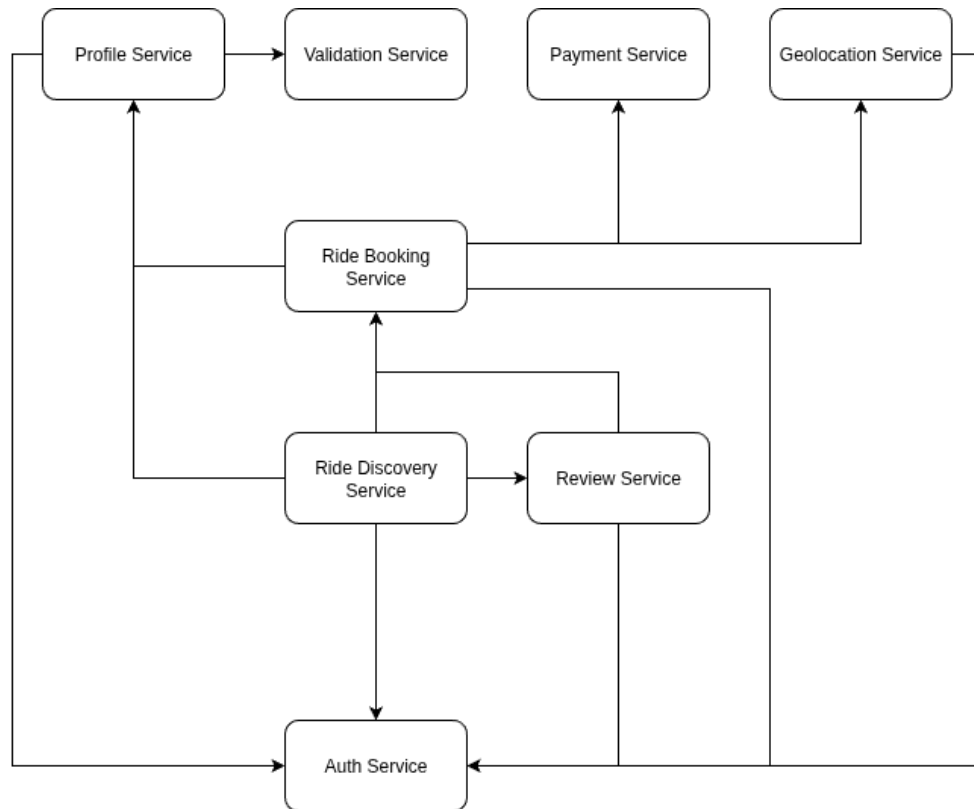
## 2.8 Geolocation Service

**Responsible:** Renan

The Geolocation Service provides geocoding to convert locations to coordinates and location autocomplete for places. This service can determine how far a passenger is from the start or end location of a ride. It primarily handles data concerning the most used locations. It is maintained as a standalone service because actions such as autocomplete generate significant traffic, and isolating it enables better scaling of computing resources without affecting other services. Additionally, this functionality can be seamlessly outsourced to services such as Google or Mapbox. It depends solely on the Authentication service to validate whether the request is legitimate.

## 2.9 Diagram

Figure 2.1 shows the architecture diagram with the eight microservices and their dependencies described above.



**Figure 2.1:** Simple Architecture Diagram